

ApiaryLens Architecture and Design Plan

Deep-dive living architecture document

Generated: 2026-07-07 03:27:20

ApiaryLens Architecture and Design Plan

Purpose

ApiaryLens is an open-source, self-hosted apiary intelligence and hive management platform. It must support a new beekeeper with one hive, a family learning together, bee clubs and mentors, extension office workflows, research use cases, and commercial growth over time.

Product Principles

- Open source first.
- Self-hosted first.
- Offline-first PWA.
- Privacy-first.
- No required paid cloud services for core operation.
- AI-assisted, not AI-required.
- SaaS-capable later, but not SaaS-dependent.
- User data belongs to the user and must be exportable.
- The product must work for one hive and scale to 100+ hives.
- The UI must be usable outdoors, on a phone, in sun glare, and with gloves.
- Documentation and task files guide coding agents before code is written.

High-Level Architecture

```
User Browser / PWA / Future Native App
|
v
Web Frontend
|
v
Backend API
|
+--> PostgreSQL
+--> Local media storage
+--> Optional S3-compatible object storage
+--> Background worker
+--> Weather provider adapters
+--> Bloom data providers
+--> Optional AI provider adapters
```

Initial Repository Strategy

The first repository is the main monorepo:

ApiaryLens/apiarylens

Initial monorepo structure:

```
apiarylens/  
  ■■■ .github/  
  ■■■ apps/  
    ■ ■■■ web/  
    ■ ■■■ api/  
    ■ ■■■ worker/  
    ■ ■■■ mobile/  
  ■■■ packages/  
    ■ ■■■ ui/  
    ■ ■■■ shared/  
    ■ ■■■ database/  
    ■ ■■■ api-client/  
    ■ ■■■ config/  
  ■■■ docs/  
  ■■■ tasks/  
  ■■■ prompts/  
  ■■■ docker/  
  ■■■ scripts/  
  ■■■ AGENTS.md  
  ■■■ README.md  
  ■■■ LICENSE  
  ■■■ CONTRIBUTING.md  
  ■■■ SECURITY.md  
  ■■■ CODE_OF_CONDUCT.md
```

Do not split into many repositories until there is operational need.

Domain Strategy

Domains owned:

- apiarylens.org
- apiarylens.com
- apiarylens.app
- apiarylens.dev

Recommended use:

- apiarylens.org - main open-source website, documentation, downloads, community.
- apiarylens.com - reserved for future commercial/company/SaaS support, redirect to .org for now.
- apiarylens.app - future PWA/app login/demo/mobile landing page, redirect to .org for now.
- apiarylens.dev - developer docs, API docs, SDK docs, plugin docs, contributor portal.

Frontend Architecture

Likely direction:

- React
- TypeScript
- Vite
- Tailwind CSS
- shadcn/ui
- PWA first
- Capacitor later for iOS/Android

- Offline local persistence using IndexedDB and/or SQLite
- MapLibre for maps
- Recharts or similar for trends

Frontend priorities:

- Large touch targets
- Fast inspection logging
- Works offline in the bee yard
- QR-code entry into a hive record
- Photo capture and upload
- Simple mode for new beekeepers
- Advanced mode for commercial/research use

Backend Architecture

Backend framework is not finalized. Candidate choices:

- FastAPI
- NestJS

The backend should provide:

- REST API first
- OpenAPI specification
- Auth and authorization
- Organization/membership model
- CRUD for apiaries, hives, inspections, queens, equipment, media, weather, bloom, harvests
- Media metadata and storage abstraction
- Background jobs for weather sync and AI analysis
- Export endpoints

Database

PostgreSQL is the likely database.

Core entities:

- Organization
- User
- Membership
- Role
- Apiary
- Hive
- HiveComponent

- Box
- Frame
- Queen
- Inspection
- InspectionObservation
- MediaAsset
- AIReview
- MiteCount
- DiseaseObservation
- Treatment
- Feeding
- Harvest
- WeatherRecord
- BloomRecord
- Plant
- Task
- ShareLink
- AuditLog

Authentication and Authorization

Start with local authentication. Design for OIDC later.

Initial options:

- Local username/password
- Optional magic link later
- Optional Google/Microsoft/GitHub OAuth later
- Optional OIDC/Keycloak later

Roles:

- Owner
- Admin
- Apiary Manager
- Inspector
- Mentor
- Read-only Viewer
- Club Member

Permissions should control access to:

- View apiary/hive

- Edit hive
- Upload media
- Create inspections
- Approve AI notes
- Manage users
- Export data

Media and AI Review

Media is core, not an add-on.

Support:

- Photos
- Videos
- Inspection images
- Brood frame images
- Queen photos
- Pest/disease photos
- Entrance photos
- Mite board photos
- Harvest photos
- Timeline/almanac view by hive

AI review principles:

- Optional and disabled by default.
- Provider-pluggable.
- Never definitive diagnosis.
- AI can flag possible observations.
- Human confirms and saves findings.

Possible providers:

- OpenAI
- Anthropic
- Local vision model
- Disabled/offline mode

Weather and Bloom Intelligence

Weather should be stored historically, not just displayed.

Track:

- High/low temperature

- Rainfall
- Humidity
- Wind
- Frost dates
- Growing degree days
- Drought
- Storms
- Heat waves
- Cold snaps
- Weather-blocked inspections
- Honey flow vs weather
- Winter consumption vs temperature

Bloom and forage data:

- Plant common name
- Scientific name
- Bloom start/end
- Nectar value
- Pollen value
- Region
- USDA zone
- Native/invasive
- Community contributed observations

Deployment

MVP deployment target:

```
docker compose up -d
```

Deployment principles:

- Self-hosted first
- No required SaaS account
- No required paid AI provider
- Local media storage first
- S3-compatible storage optional later

Future:

- Kubernetes
- Helm charts
- Managed SaaS

- Hosted demo

Roadmap

Phase 0: Project Foundation

- GitHub org setup
- Domains owned
- Repo foundation
- Docs
- AGENTS.md
- Tasks
- ADRs

Phase 1: MVP Foundation

- Auth
- Organizations
- Users
- Apiaries
- Hives
- PWA shell
- API
- PostgreSQL
- Docker Compose

Phase 2: Hive Records

- Inspections
- Queens
- Boxes
- Frames
- Photos/videos
- QR codes

Phase 3: Health and Production

- Varroa
- Diseases/pests
- Treatments
- Feeding
- Honey/wax harvest
- Reports

Phase 4: Weather and Bloom Intelligence

- Forecasts
- Historical weather
- Weather patterns
- Bloom calendar
- Forage database

Phase 5: Sharing and Club Mode

- Mentor sharing
- Family sharing
- Bee club organizations
- Read-only reports

Phase 6: AI Assistance

- AI image review
- AI summaries
- AI risk flags
- Provider abstraction

Phase 7: Native Apps

- Capacitor wrapper
- iOS
- Android
- Push notifications
- Camera integration

Phase 8: Sensors and Integrations

- Hive scales
- Sensors
- MQTT
- Home Assistant

Phase 9: Commercial / Research Scale

- Route planning
- Employee roles
- Pollination contracts
- Research exports
- SaaS option